Adam

April 28, 2026 6:06 PM

when training neural networks, were minimizing the loss function using gradients:

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$

This approach is not the best because different parameters work better with different learning rates.

Adam improves things by giving each parameter their own learning rate and smoothing learning.

Adam formula:

$\theta_t \longrightarrow$ Parameters at step t.

$\theta_{t-1} \longrightarrow$ Parameters at step t-1.

$g_t \longrightarrow$ gradient(s) at step t (Adam does not calculate this in my implementation)

$m_t \longrightarrow$ momentum : smoothed/Averaged gradient

$m_{t-1} \longrightarrow$ previous momentum

$\beta_1 \longrightarrow$ momentum decay rate: usually 0.9 . close to 1 means remember a lot of history

$v_t \longrightarrow$ Variance/second moment: Average of squared gradients

$v_{t-1} \longrightarrow$ Previous variance estimate

$g_t^2 \longrightarrow$ Squared gradient

$\beta_2 \longrightarrow$ Variance decay rate: usually 0.999

$\hat{m}_t \longrightarrow$ Bias corrected momentum

$\hat{v}_t \longrightarrow$ Bias corrected variance

$t \longrightarrow$ step count

$\eta \rightarrow$ learning rate

$\epsilon \rightarrow 10^{-8}$. tiny constant to prevent division by 0.

The core component: "Forgetful" Average

The core component: "Forgetful" Average

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t$$

the momentum is adjusted using a large portion of the previous gradient average, and a small portion of the current gradient.

Basically we have an average momentum, and each gradient is able to make small adjustments to it.

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

The average magnitude is adjusted using a small portion of the square of the current gradient

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

At the start, $v_t = m_t = 0$, so estimates are biased towards zero. we fix that by doing bias correction.

<u>Parameter update:</u>

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

we divide the momentum by $\sqrt{\hat{v}_t}$ because if a parameter has large gradients consistently, v_t becomes large and steps become small. if gradients are small, v_t becomes small and steps become large. this gives per-parameter learning rates automatically.